Assignment 4: Parallel Python for HPC

Matthew Lochridge

March 2026

1 Inverse Transform Sampling (Lorentzian)

We explore several methods of parallelism in Python by random sampling from the Lorentzian distribution, having the probability density function (PDF)

$$p(x) = \frac{\Gamma/\pi}{\Gamma^2 + x^2} \tag{1}$$

centered at $x = 0$, where Γ is the half-width at half-maximum (HWHM).

By integrating the PDF from $-\infty$ to arbitrary x, we find the normalized cumulative distribution function (CDF)

$$F(x) = \int_{-\infty}^{x} p(x')dx' = \frac{1}{\pi}\tan^{-1}\left(\frac{x}{\Gamma}\right) + \frac{1}{2} \tag{2}$$

with $\lim_{x\to\infty} F(x) = 1$.

Since $F(x) \in (0, 1)$, we replace the CDF with samples from a uniform random distribution $u \in (0, 1)$, and Lorentzian samples x are generated from the inverse CDF

$$F^{-1}(u) = \Gamma \tan\left(\pi\left(u - \frac{1}{2}\right)\right) = -\frac{\Gamma}{\tan(\pi u)} \tag{3}$$

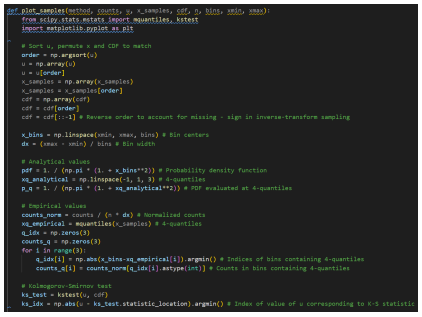
Now $x = F^{-1}(u) \in \mathbb{R}$ for $u \in (0, 1)$, as desired. Since the PDF is symmetric about $x = 0$, we may omit the negative sign in Eqn. (3) without consequence.

In the following codes, we set $\Gamma = 1$.

2 Base NumPy Implementation

The provided code for the base NumPy implementation lorenzian_histogram() was modified to compute the CDF as in Eqn. [2], and to return u, x, and the CDF in addition to the histogram data. These additional data were used in the analysis to perform the Kolmogorov-Smirnov (K-S) test. Just prior to sampling, the current time is used to seed the RNG.

The analysis function plot_samples(), shown in Fig. [1], was written to visualize the empirical histogram with 4-quantiles against the analytical PDF, as well as the K-S test of u against the CDF.



```
def plot_samples(method, counts, u, x_samples, cdf, n, bins, xmin, xmax):
    from scipy.stats.mstats import mquantiles, kstest
    import matplotlib.pyplot as plt

    # Sort u, permute x and CDF to match
    order = np.argsort(u)
    u = np.array(u)
    u = u[order]
    x_samples = np.array(x_samples)
    x_samples = x_samples[order]
    cdf = np.array(cdf)
    cdf = cdf[order]
    cdf = cdf[::-1] # Reverse order to account for missing - sign in inverse-transform sampling

    x_bins = np.linspace(xmin, xmax, bins) # Bin centers
    dx = (xmax - xmin) / bins # Bin width

    # Analytical values
    pdf = 1. / (np.pi * (1. + x_bins**2)) # Probability density function
    xq_analytical = np.linspace(-1, 1, 3) # 4-quantiles
    p_q = 1. / (np.pi * (1. + xq_analytical**2)) # PDF evaluated at 4-quantiles

    # Empirical values
    counts_norm = counts / (n * dx) # Normalized counts
    xq_empirical = mquantiles(x_samples) # 4-quantiles
    q_idx = np.zeros(3)
    counts_q = np.zeros(3)
    for i in range(3):
        q_idx[i] = np.abs(x_bins-xq_empirical[i]).argmin() # Indices of bins containing 4-quantiles
        counts_q[i] = counts_norm[q_idx[i].astype(int)] # Counts in bins containing 4-quantiles

    # Kolmogorov-Smirnov test
    ks_test = kstest(u, cdf)
    ks_idx = np.abs(u - ks_test.statistic_location).argmin() # Index of value of u corresponding to K-S statistic
```

(a)

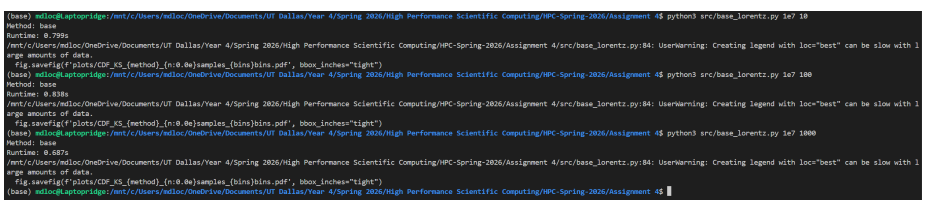
```
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 8), sharex=True)
# Visualize u and CDF with K-S statistic
ax1.semilogy(np.abs(x_samples), u, label='Uniform(0,1)')
ax1.semilogy(np.abs(x_samples), cdf, color='orange', linestyle='dashed', label='CDF')
ax1.annotate(text='',
             xy=(np.abs(x_samples[ks_idx]), ks_test.statistic_location),
             xytext=(np.abs(x_samples[ks_idx]), ks_test.statistic_location + ks_test.statistic_sign*ks_test.statistic),
             arrowprops=dict(arrowstyle='->', connectionstyle='arc3', color='k'))
ax1.set_title(f'{method} method: K-S Test')
ax1.legend(handles=[plt.Line2D([], [], label='Uniform(0,1)'),
                    plt.Line2D([], [], color='orange', linestyle='dashed', label='CDF'),
                    plt.Line2D([0], [0], color='black', marker='|', linestyle='None', label='KS Statistic'],
           labels=['Uniform(0,1)', 'CDF', 'K-S Statistic',
           loc='upper left')
# Visualize residuals of CDF - u
ax2.semilogy(x_samples, cdf - u) # Reverse order of CDF to account for missing - sign in inverse-transform sampling
ax2.annotate(text='',
             xy=(np.abs(x_samples[ks_idx]), ks_test.statistic_location - u[ks_idx]),
             xytext=(np.abs(x_samples[ks_idx]), ks_test.statistic_location - u[ks_idx] + ks_test.statistic_sign*ks_test.statistic),
             arrowprops=dict(arrowstyle='->', connectionstyle='arc3', color='k'))
ax2.set_title(f'{method} method: K-S Test')
ax2.legend(handles=[plt.Line2D([], [], label='CDF - Uniform(0,1)'),
                    plt.Line2D([0], [0], color='black', marker='|', linestyle='None', label='KS Statistic']),
           labels=['CDF - Uniform(0,1)', 'K-S Statistic'])
ax2.set_xlabel('|x|')
ax2.set_title('Residuals of Transformed Samples')
fig.savefig(f'plots/CDF_KS_{method}_{n:0.0e}samples_{bins}bins.pdf', bbox_inches='tight')

# Plot histogram
plt.figure(figsize=(12, 8))
plt.bar(x_bins, counts_norm, label='Samples')
plt.plot(x_bins, pdf, label='PDF', color='r')
plt.plot(xq_analytical, p_q, 'o', label='Analytical 4-quantiles', color='r')
plt.plot(xq_empirical, counts_q, 'o', label='Empirical 4-quantiles', color='k')
plt.xlabel('x')
plt.ylabel('p(x)')
plt.title(f'{method} method: Lorentzian Distribution with {n:0.0e} Samples and {bins} Bins')
plt.legend()
plt.savefig(f'plots/hist_{method}_{n:0.0e}samples_{bins}bins.pdf', bbox_inches='tight')
```

(b)

Figure 1: a: Beginning of plot_samples() function, showing analysis computations. b: End of plot_samples() function, showing figure generation.

A main function was written to optionally take in command line arguments for the number of samples, number of bins, and domain boundaries for bins; to call lorentz_histogram(), timing its execution with time.perf_counter(); and call plot_samples() for analysis. The terminal log for testing the NumPy method locally is shown in Fig. [2]. For 10, 100, or 1000 bins, the execution time is about the same.



```
(base) mdloc@Laptopridge:/mnt/c/Users/mdloc/OneDrive/Documents/UT Dallas/Year 4/Spring 2026/High Performance Scientific Computing/HPC-Spring-2026/Assignment 4$ python3 src/base_lorentz.py 1e7 10
Method: base
Runtime: 0.799s
/mnt/c/Users/mdloc/OneDrive/Documents/UT Dallas/Year 4/Spring 2026/High Performance Scientific Computing/HPC-Spring-2026/Assignment 4/src/base_lorentz.py:84: UserWarning: Creating legend with loc="best" can be slow with l
arge amounts of data.
  fig.savefig(f'plots/CDF_KS_{method}_{n:0.0e}samples_{bins}bins.pdf', bbox_inches="tight")
(base) mdloc@Laptopridge:/mnt/c/Users/mdloc/OneDrive/Documents/UT Dallas/Year 4/Spring 2026/High Performance Scientific Computing/HPC-Spring-2026/Assignment 4$ python3 src/base_lorentz.py 1e7 100
Method: base
Runtime: 0.838s
/mnt/c/Users/mdloc/OneDrive/Documents/UT Dallas/Year 4/Spring 2026/High Performance Scientific Computing/HPC-Spring-2026/Assignment 4/src/base_lorentz.py:84: UserWarning: Creating legend with loc="best" can be slow with l
arge amounts of data.
  fig.savefig(f'plots/CDF_KS_{method}_{n:0.0e}samples_{bins}bins.pdf', bbox_inches="tight")
(base) mdloc@Laptopridge:/mnt/c/Users/mdloc/OneDrive/Documents/UT Dallas/Year 4/Spring 2026/High Performance Scientific Computing/HPC-Spring-2026/Assignment 4$ python3 src/base_lorentz.py 1e7 1000
Method: base
Runtime: 0.687s
/mnt/c/Users/mdloc/OneDrive/Documents/UT Dallas/Year 4/Spring 2026/High Performance Scientific Computing/HPC-Spring-2026/Assignment 4/src/base_lorentz.py:84: UserWarning: Creating legend with loc="best" can be slow with l
arge amounts of data.
  fig.savefig(f'plots/CDF_KS_{method}_{n:0.0e}samples_{bins}bins.pdf', bbox_inches="tight")
(base) mdloc@Laptopridge:/mnt/c/Users/mdloc/OneDrive/Documents/UT Dallas/Year 4/Spring 2026/High Performance Scientific Computing/HPC-Spring-2026/Assignment 4$
```

Figure 2: Terminal log for base NumPy runs, showing execution time of random sampling.

Fig. [3] shows the results for the NumPy method for 10^7 samples and 10, 100, and 1000 bins. The PDF was evaluated at the bin centers rather than for millions of samples; in the case of 10 bins, the lack of a central bin flattens the PDF curve, but the true analytical median is shown. We can see that the empirical quantiles approach the analytical quantiles as the number of bins increases. Additionally, we see that applying the CDF function to the inverse-transformed samples reproduces the uniform random samples with high accuracy.

This pipeline was adapted for the following methods as well, including additional command line arguments for parallelism. NumPy's SeedSequence function was used to generate statistically independent seeds that were used to

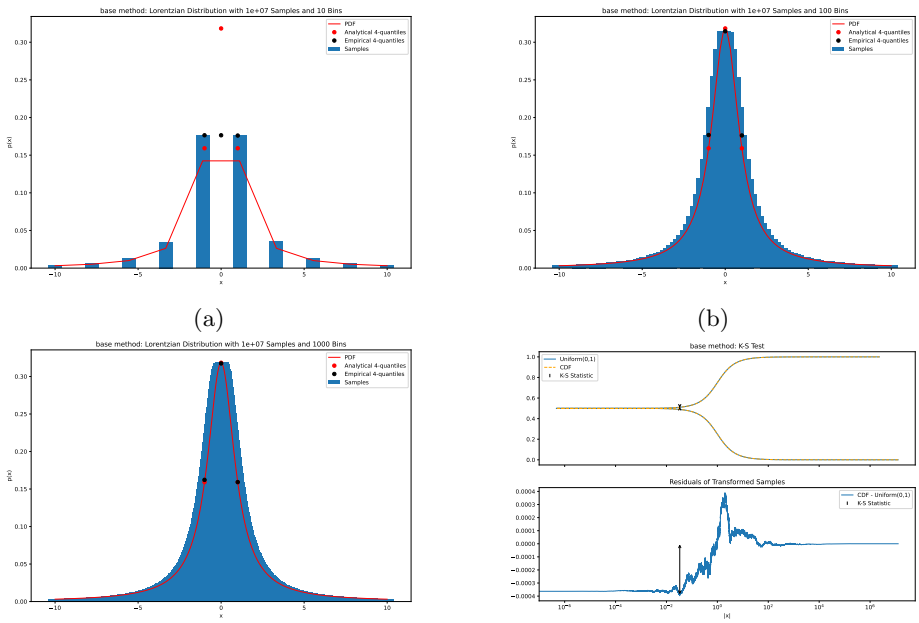


Figure 3: Sampling results from base NumPy method. a-c: Empirical histograms with 10^7 samples and 10, 100, and 1000 bins, compared with the analytical PDF. d: Comparison of CDF and Uniform(0,1) with K-S test.

initialize the RNG for each parallel processor. The provided Slurm batch script was modified to iterate through each CPU method, number of processors, and weak/strong scaling on Ganymede2. Tests were run with $p = 1$, 2, 4, 8, 16, and 32 parallel processors (since Joblib could not complete the job with 64 processors within 10 minutes). For strong scaling, $N = 10^7$ samples were used, and for weak scaling, $p \times 10^7$ samples were used. Each unique case was run 5 times, and the average runtimes were taken for scaling analyses. Error bars were produced by propagating the standard error in the average runtimes. Batch scripts are shown in Fig. [4].

Since varying the bins between 10, 100, and 1000 did not significantly affect the runtime for a fixed number of samples, I report results for the following methods using 100 bins only for simplicity. Histograms are shown only for $N = 10^7$ samples, and quantiles and K-S tests are omitted; the $O(N)$ arrays used for the quantiles and K-S test are not stored in the parallel methods, so the parallel performance is analyzed strictly for RNG.

```bash
#!/bin/bash
#SBATCH --job-name=hw4
#SBATCH --partition=cpu-preempt
#SBATCH --nodes=1 # nodes
#SBATCH --ntasks=64 # total ranks
#SBATCH --cpus-per-task=1 # CPUs per rank
#SBATCH --time=00:10:00 # walltime
#SBATCH --exclusive # avoid interference
#SBATCH --mail-user=md1220000@utdallas.edu

set -euo pipefail

# Avoid oversubscription by BLAS/NumPy threads
export OMP_NUM_THREADS=1
export MKL_NUM_THREADS=1

METHOD=$1
SCALING=$2
N=${3:-10000000}
P=64

if [[ "$SCALING" == "weak" ]]; then
    N=$((N * P))
fi

if [[ "$METHOD" == "mpi" ]]; then
    mpiexec -n $P python3 "src/${METHOD}_lorentz.py" $SCALING $N
else
    prun python3 "src/${METHOD}_lorentz.py" $SCALING $N $P
fi
```

(a)

```bash
#!/bin/bash

MAX_PARALLELISM=${1:-64}
N_BASE=${2:-10000000}
METHODS=("async" "dask" "joblib" "mp" "mpi" "mpire" "ppe" "thread")
SCALING_MODES=("weak" "strong")

for METHOD in "${METHODS[@]}"; do
    for SCALING in "${SCALING_MODES[@]}"; do
        # for ((p=1; p<=$MAX_PARALLELISM; p*=2)); do
            # if [[ $p -gt $MAX_PARALLELISM ]]; then
                # break
            # fi
        sbatch "run_p128.slurm" $METHOD $SCALING $N_BASE
        # done
    done
done
squeue -u md1220000
```

(b)

Figure 4: a: Slurm job script with 64 tasks. b: Batch script to iteratively run jobs.

3 Threading

The provided code was modified to optionally take in the number of threads as an argument and to independently seed each thread's RNG. A main function was written to mimic the base pipeline. The code is shown in Fig. [5], and the resulting histogram is shown in Fig. [6].

Batches were run on Ganymede2 to test weak and strong scaling up to 32 threads. The runtime, speedup, and efficiency plots are shown in Fig. [7]. A table summarizing the strong scaling speedup vs. concurrency is shown in Fig. [8].

The speedup for the threading method appears to peak around 16 cores, due to increasing overhead with the GIL for increased threading. For low parallelism and $N = 10^7$ samples, Amdahl's law appears to be a much better fit for strong scaling than Gustafson's law.

4 Multiprocessing

The provided code was modified to optionally take in the number of cores as an argument and to independently seed each core's RNG. A main function was written to mimic the base pipeline. The code is shown in Fig. [9], and the resulting histogram is shown in Fig. [10].

Batches were run on Ganymede2 to test weak and strong scaling up to 32 cores. The runtime, speedup, and efficiency plots are shown in Fig. [11]. A table summarizing the strong scaling speedup vs. concurrency is shown in Fig. [12].

```python
def add_chunk(seed, n, counts, u, x, cdf, lock, bins=100, xmin=-10, xmax=10):
    """
    Generate n samples and add to global counts.
    """
    np.random.default_rng(seed)
    local_counts, local_u, local_x, local_cdf = lorentzian_histogram(n, bins, xmin, xmax)
    # Acquire lock to merge partial counts into global
    with lock:
        counts += local_counts
        u.extend(local_u)
        x.extend(local_x)
        cdf.extend(local_cdf)

def run_threaded(seeds, n, n_threads=4, bins=100, xmin=-10, xmax=10):
    """
    Run the Lorentzian sampling in parallel using threads.
    """
    # Split n samples among processes
    chunks = [n // n_threads] * np.ones(n_threads, dtype=int)
    chunks[:n % n_threads] += 1 # Distribute remainder
    threads = [None] * n_threads # Thread list
    counts = np.zeros(bins) # Global counts
    u = [] # Global uniform samples
    x = [] # Global transformed samples
    cdf = [] # Global CDF
    lock = threading.Lock() # Lock for global data
    for i in range(n_threads):
        t = threading.Thread(target=add_chunk, args=(seeds[i], chunks[i], counts, u, x, cdf, lock, bins, xmin, xmax))
        t.start() # Start thread
        threads[i] = t
    for t in threads:
        t.join() # Wait for all threads to finish
    return counts, u, x, cdf
```

(a)

```python
if __name__ == "__main__":

    method = 'threading'
    scaling = sys.argv[1] # 'weak' or 'strong'
    n = int(3e7) # Default number of samples
    n_threads = 4 # Default number of threads
    bins = 100 # Default number of bins
    xmin = -10 # Default xmin
    xmax = 10 # Default xmax

    # Parse command line arguments, if provided
    if len(sys.argv) < 2:
        print("Usage: python thread_lorentz.py <weak|strong> <n> <n_tasks> <bins> <xmin> <xmax> <n_subchunks>")
        print("Error: Scaling not specified.")
        sys.exit(1)
    if len(sys.argv) >= 3:
        n = int(float(sys.argv[2])) # Overwrite default n
    if len(sys.argv) >= 4:
        n_threads = int(float(sys.argv[3])) # Overwrite default n_threads
    if len(sys.argv) >= 5:
        bins = int(float(sys.argv[4])) # Overwrite default bins
    if len(sys.argv) >= 6:
        xmin = int(float(sys.argv[5])) # Overwrite default xmin
    if len(sys.argv) == 7:
        xmax = int(float(sys.argv[6])) # Overwrite default xmax
    if len(sys.argv) >= 8:
        print("Usage: python thread_lorentz.py <weak|strong> <n> <n_threads> <bins> <xmin> <xmax>")
        print("Additional arguments will be ignored.")

    seed_seq = np.random.SeedSequence(int(time()))
    seeds = seed_seq.spawn(n_threads)

    start_time = perf_counter()
    counts, u, x, cdf = run_threaded(seeds, n, n_threads, bins, xmin, xmax)
    end_time = perf_counter()
    print(f'Method: {method}')
    print(f'Runtime: {end_time-start_time:e}s')
    with open(f"results/{scaling}_scaling_{method}.txt", "a", encoding="utf-8") as f:
        f.write(f"{n_threads}\t{end_time-start_time}\n")
    if n_threads == 1 and scaling == 'strong':
        plot_samples(method, counts, u, x, cdf, n, bins, xmin, xmax)
```

(b)

Figure 5: a: Modified threading algorithm. b: Main function.

Multiprocessing was expected to scale better than threading due to avoiding the GIL, though there appears to be fluctuation in the performance. This may wash out by averaging the timing results of several runs. Qualitatively, we see that Amdahl's law is still the better fit for scaling in this regime.

5 ProcessPoolExecutor

The provided code was modified to optionally take in the number of workers as an argument and to independently seed each worker's RNG. A main function was written to mimic the base pipeline. The code is shown in Fig. [13], and the resulting histogram is shown in Fig. [14].

Batches were run on Ganymede2 to test weak and strong scaling up to 32 workers. The runtime, speedup, and efficiency plots are shown in Fig. [15]. A table summarizing the strong scaling speedup vs. concurrency is shown in Fig. [16].

ProcessPoolExecutor not only waits to merge the results of parallel calculations, but it also waits to execute calculations synchronously. This results in much better performance and less fluctuation when scaling than seen with multiprocessing. Amdahl's law appears to fit the data better than Gustafson's law in this regime as well.

6 AsyncIO

The provided code was modified to optionally take in the number of tasks as an argument and to independently seed each task's RNG. A main function was written to mimic the base pipeline. The code is shown in Fig. [17], and the resulting histogram is shown in Fig. [18].

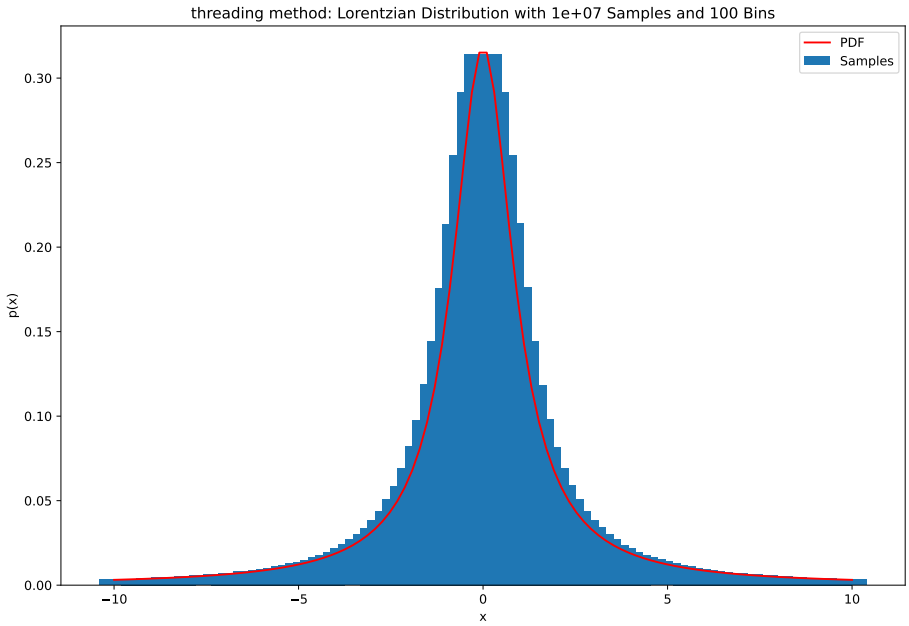


Figure 6: Sampling results from the threading method.

Batches were run on Ganymede2 to test weak and strong scaling up to 32 tasks. The runtime, speedup, and efficiency plots are shown in Fig. [19]. A table summarizing the strong scaling speedup vs. concurrency is shown in Fig. [20].

Due to the GIL as well as waiting to merge results, AsyncIO scales poorly for CPU-bound tasks. We see the performance start to pick up by 32 tasks, and before then the communication overhead dominates. For the low parallelism tested, this results in nonsensical serial fractions greater than one, though it's possible that this fit could be improved by averaging multiple runs.

7 Dask

The provided code was modified to optionally take in the number of tasks as an argument and to independently seed each task's RNG. The default single-threaded scheduler was overridden to make use of multiple threads. A main function was written to mimic the base pipeline. The code is shown in Fig. [21], and the resulting histogram is shown in Fig. [22].

Batches were run on Ganymede2 to test weak and strong scaling up to 32 tasks. The runtime, speedup, and efficiency plots are shown in Fig. [23]. A table summarizing the strong scaling speedup vs. concurrency is shown in Fig. [24].

We see similar poor scaling for delayed threaded Dask as with AsyncIO,

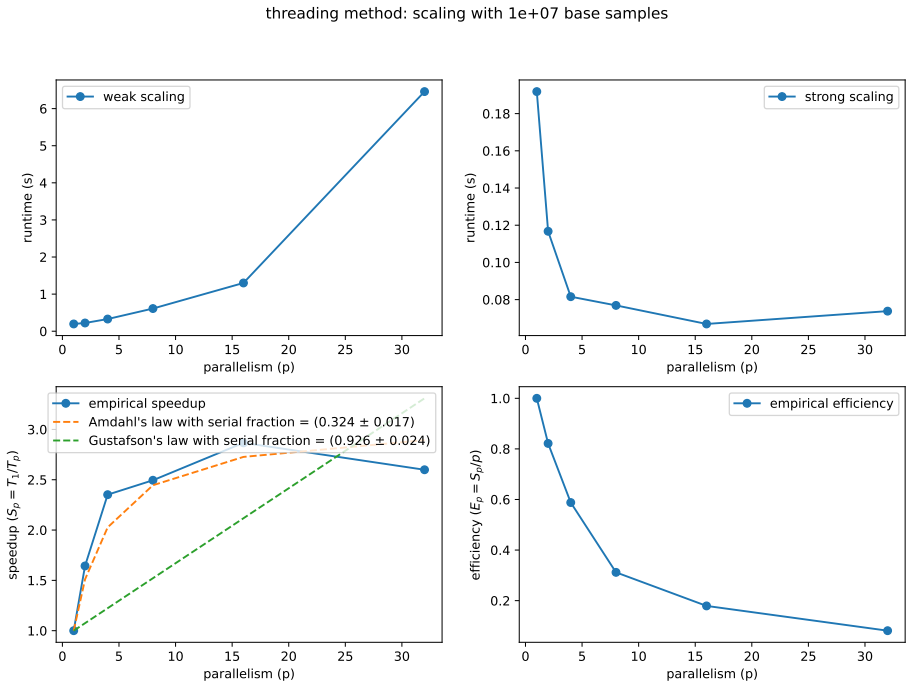


Figure 7: Scaling analysis for the threading method. Speedup and efficiency correspond to strong scaling.

including nonsensical serial fractions greater than 1. Though running delayed Dask with a process scheduler would probably improve these by avoiding overhead caused by the GIL. Dynamic chunking may also improve performance, though this was not implemented.

8 Numba

The provided Numba code was modified the most. Since the atomic package for CPU-based Numba was deprecated, I resorted to using Cuda, since that alternative affected the syntax the least. RNG seeds were generated for each thread using xoroshiro128p packages. The number of blocks per grid and threads per block are optionally read in as arguments. The code is shown in Fig. [25]. The resulting histogram is shown in Fig. [26].

Batches were run locally (since Ganymede2 does not provide free access to GPUs) to test weak and strong scaling up to 32 threads for 1 block. The runtime, speedup, and efficiency plots are shown in Fig. [27]. A table summarizing the strong scaling speedup vs. concurrency is shown in Fig. [28].

Numba with Cuda demonstrates the best scaling so far, though we are now comparing GPU performance to CPU performance. Amdahl's law is a very close

Concurrency	Speedup
1.0	1.0
2.0	1.6432938530600916
4.0	2.351648266589552
8.0	2.494867770268762
16.0	2.8668067848377103
32.0	2.5991772408677374

Figure 8: Table of strong scaling data for the threading method.

```python
def initializer():
    return np.random.default_rng(int(time()))

def run_multiproc(n, n_cores=4, bins=100, xmin=-10, xmax=10):
    """
    Run the Lorentzian sampling in parallel using processes.
    """
    # Split n samples among processes
    chunks = (n // n_cores) * np.ones(n_cores, dtype=int)
    chunks[:n % n_cores] += 1 # Distribute remainder
    # Use partial function to reset default arguments (bins, xmin, xmax)
    from functools import partial
    lorentzian_hist_func = partial(lorentzian_histogram, bins=bins, xmin=xmin, xmax=xmax)
    # Use Pool to distribute chunks to processes
    with multiprocessing.Pool(n_cores, initializer) as pool:
        results = pool.map(lorentzian_hist_func, chunks)
    # Unpack results
    counts = np.zeros(bins)
    u = []
    x = []
    cdf = []
    for r in results:
        counts += r[0]
        u.extend(r[1])
        x.extend(r[2])
        cdf.extend(r[3])
    return counts, u, x, cdf
```

(a)

```python
if __name__ == "__main__":

    method = 'multiprocessing'
    scaling = sys.argv[1] # 'weak' or 'strong'
    n = int(1e7) # Default number of samples
    n_cores = 4 # Default number of cores
    bins = 100 # Default number of bins
    xmin = -10 # Default xmin
    xmax = 10 # Default xmax

    # Parse command line arguments, if provided
    if len(sys.argv) < 2:
        print("Usage: python mp_lorentz.py <weak|strong> <n> <n_tasks> <bins> <xmin> <xmax> <n_subchunks>")
        print("Error: Scaling not specified.")
        sys.exit(1)
    if len(sys.argv) >= 3:
        n = int(float(sys.argv[2])) # Overwrite default n
    if len(sys.argv) >= 4:
        n_cores = int(float(sys.argv[3])) # Overwrite default n_cores
    if len(sys.argv) >= 5:
        bins = int(float(sys.argv[4])) # Overwrite default bins
    if len(sys.argv) >= 6:
        xmin = int(float(sys.argv[5])) # Overwrite default xmin
    if len(sys.argv) >= 7:
        xmax = int(float(sys.argv[6])) # Overwrite default xmax
    if len(sys.argv) >= 8:
        print("Usage: python mp_lorentz.py <weak|strong> <n> <n_cores> <bins> <xmin> <xmax>")
        print("Additional arguments will be ignored.")

    start_time = perf_counter()
    counts, u, x, cdf = run_multiproc(n, n_cores, bins, xmin, xmax)
    end_time = perf_counter()
    print(f'Method: {method}')
    print(f'Runtime: {end_time-start_time:e}s')
    with open(f"results/{scaling}_scaling_{method}.txt", "a", encoding="utf-8") as f:
        f.write(f"{n_cores}\t{end_time-start_time}\n")
    if n_cores == 1 and scaling == 'strong':
        plot_samples(method, counts, u, x, cdf, n, bins, xmin, xmax)
```

(b)

Figure 9: a: Modified multiprocessing algorithm. b: Main function.

fit to the strong scaling data. For the weak scaling, we notice that performance is suboptimal for low parallelism.

9 Joblib

The provided code was modified to optionally take in the number of jobs as an argument and to independently seed each job's RNG. The default processes scheduler was used. A main function was written to mimic the base pipeline. The code is shown in Fig. [29], and the resulting histogram is shown in Fig. [30].

Batches were run on Ganymede2 to test weak and strong scaling up to 32 jobs. The runtime, speedup, and efficiency plots are shown in Fig. [31]. A table summarizing the strong scaling speedup vs. concurrency is shown in Fig. [32].

Joblib shows the same poor performance and nonsensical serial fractions as

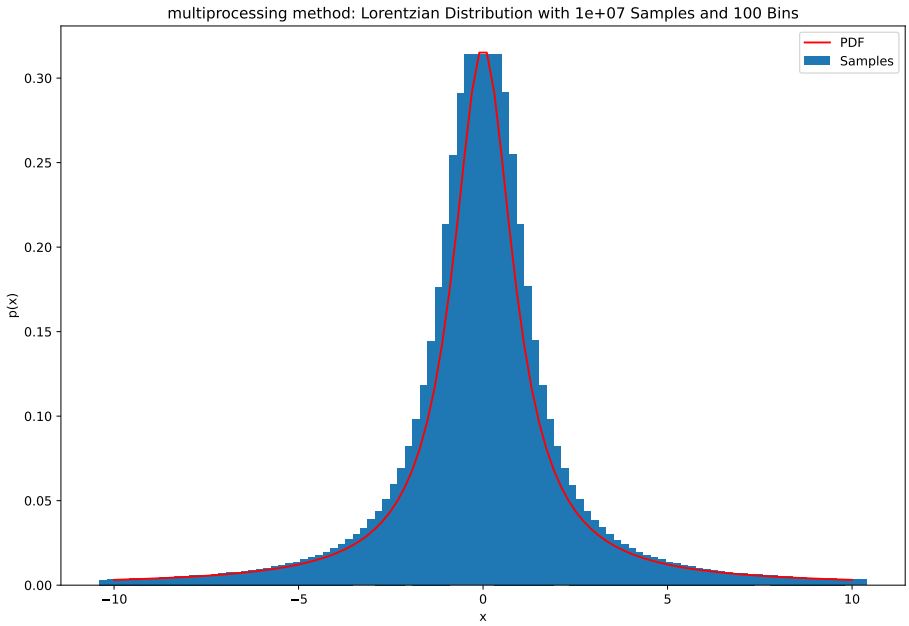


Figure 10: Sampling results from the multiprocessing method.

seen with AsyncIO and Dask. This could potentially be improved by dynamic chunking, which was not implemented.

10 mpire

The provided code was modified to optionally take in the number of jobs as an argument and to independently seed each job's RNG. The default processes scheduler was used. A main function was written to mimic the base pipeline. The code is shown in Fig. [33], and the resulting histogram is shown in Fig. [34].

Batches were run on Ganymede2 to test weak and strong scaling up to 32 jobs. The runtime, speedup, and efficiency plots are shown in Fig. [35]. A table summarizing the strong scaling speedup vs. concurrency is shown in Fig. [36].

With mpire, we see Amdahl-like behavior for low parallelism, and a sharp drop in performance for 32 jobs. It is possible that the overhead incurred by dynamic chunking impacts performance with more jobs in communication.

11 MPI

The provided code was modified to optionally take in the number of samples, number of bins, and bin domain boundaries as arguments. The code is shown

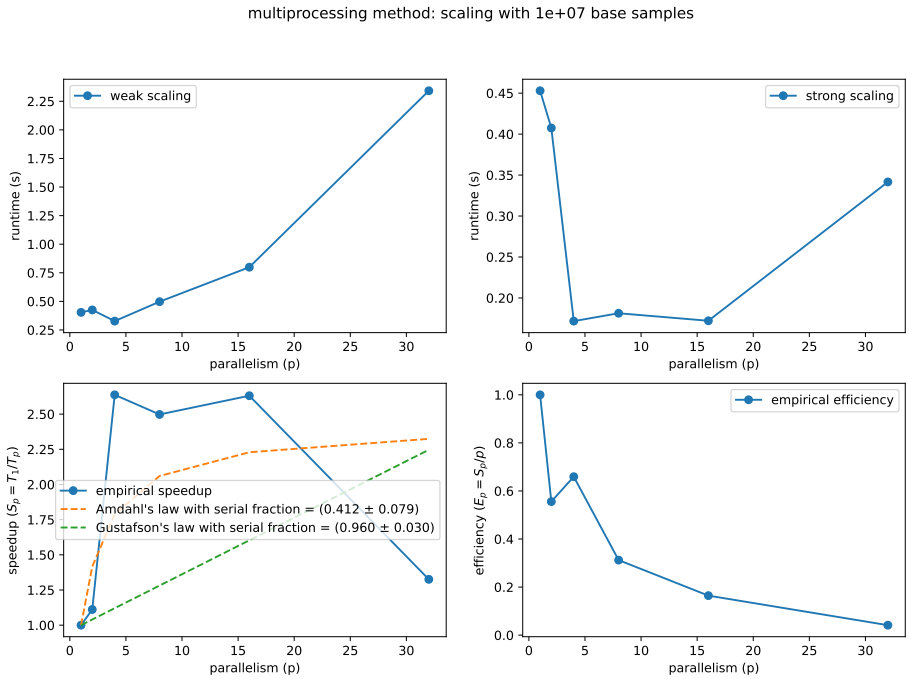


Figure 11: Scaling analysis for the multiprocessing method. Speedup and efficiency correspond to strong scaling.

in Fig. [37], and the resulting histogram is shown in Fig. [38].

Batches were run on Ganymede2 to test weak and strong scaling up to 32 ranks. The runtime, speedup, and efficiency plots are shown in Fig. [39]. A table summarizing the strong scaling speedup vs. concurrency is shown in Fig. [40].

The MPI method shows the best scaling and absolute runtime performance of any of the CPU methods. In the regime initially tested, the data closely fits with Amdahl's law.

12 Final HPC Run with MPI

The final HPC scaling run was carried out on Ganymede2 using MPI with $N = 10^9$ samples, including tests with 64 and 128 ranks. The scaling plots are shown in Fig. [41], and the table of concurrency vs. strong scaling speedup is shown in Fig. [42].

In this run we see a sharp dip in performance for 64 ranks (though this may be smoothed out by averaging multiple runs). However, the strong scaling speedup at 128 ranks is very close to the speedup at 32 ranks, implying that Amdahl's law is still the better fit for fixed N.

Concurrency	Speedup
1.0	1.0
2.0	1.1114284116547015
4.0	2.6371477309603946
8.0	2.4976721365978163
16.0	2.630717726119452
32.0	1.3257743885792694

Figure 12: Table of strong scaling data for the multiprocessing method.

```python
def initializer():
    return np.random.default_rng(int(time()))

def run_ppe(n, max_workers=4, bins=100, xmin=-10, xmax=10):
    """
    Run the Lorentzian sampling in parallel using ProcessPoolExecutor.
    """
    chunks = (n // max_workers) * np.ones(max_workers, dtype=int) # Split n samples among workers
    chunks[:n % max_workers] += 1 # Distribute remainder
    with ProcessPoolExecutor(max_workersmax_workers, initializer=initializer) as executor:
        futures = [executor.submit(lorentzian_histogram, chunk, bins, xmin, xmax) for chunk in chunks]
        results = [f.result() for f in futures] # Collect results
    # Unpack results
    counts = np.zeros(bins)
    u = []
    x = []
    cdf = []
    for r in results:
        counts += r[0]
        u.extend(r[1])
        x.extend(r[2])
        cdf.extend(r[3])
    return counts, u,  x, cdf
```

(a)

```python
if __name__ == "__main__":

    method = 'ProcessPoolExecutor'
    scaling = sys.argv[1] # 'weak' or 'strong'
    n = int(1e7) # Default number of samples
    max_workers = 4 # Default number of workers
    bins = 100 # Default number of bins
    xmin = -10 # Default xmin
    xmax = 10 # Default xmax

    # Parse command line arguments, if provided
    if len(sys.argv) < 2:
        print("Usage: python ppe_lorentz.py <weak|strong> <n> <n_tasks> <bins> <xmin> <xmax> <n_subchunks>")
        print("Error: Scaling not specified.")
        sys.exit(1)
    if len(sys.argv) >= 3:
        n = int(float(sys.argv[2])) # Overwrite default n
    if len(sys.argv) >= 4:
        max_workers = int(float(sys.argv[3])) # Overwrite default max_workers
    if len(sys.argv) >= 5:
        bins = int(float(sys.argv[4])) # Overwrite default bins
    if len(sys.argv) >= 6:
        xmin = int(float(sys.argv[5])) # Overwrite default xmin
    if len(sys.argv) == 7:
        xmax = int(float(sys.argv[6])) # Overwrite default xmax
    if len(sys.argv) >= 8:
        print("Usage: python ppe_lorentz.py <weak|strong> <n> <max_workers> <bins> <xmin> <xmax>")
        print("Additional arguments will be ignored.")

    start_time = perf_counter()
    counts, u, x, cdf = run_ppe(n, max_workers, bins, xmin, xmax)
    end_time = perf_counter()
    print(f'Method: {method}')
    print(f'Runtime: {end_time-start_time:.6}s')
    with open(f"results/{scaling}_scaling_{method}.txt", "a", encoding="utf-8") as f:
        f.write(f"{max_workers}\t{end_time-start_time}\n")
    if max_workers == 1 and scaling == 'strong':
        plot_samples(method, counts, u, x, cdf, n, bins, xmin, xmax)
```

(b)

Figure 13: a: Modified ProcessPoolExecutor algorithm. b: Main function.

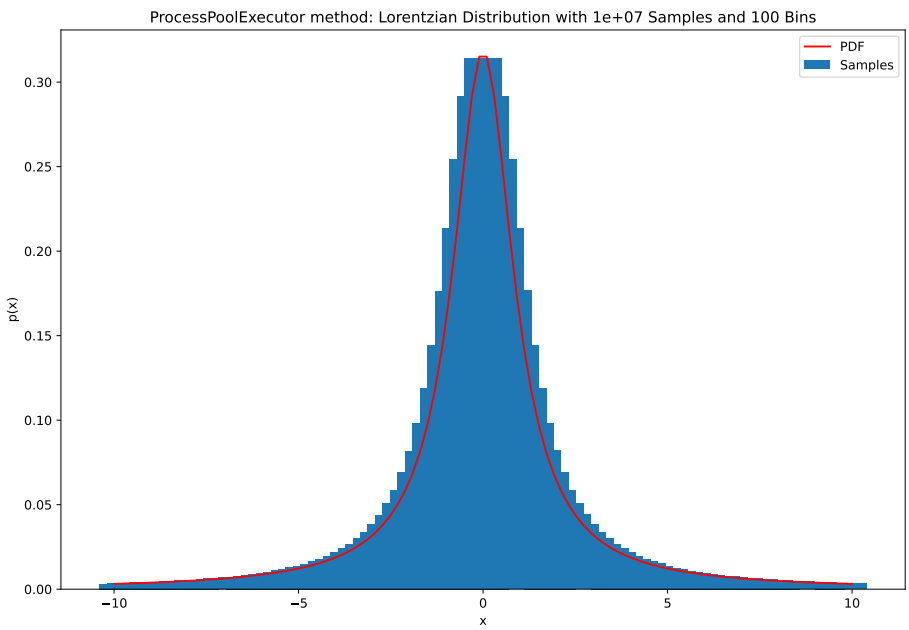


Figure 14: Sampling results from the ProcessPoolExecutor method.

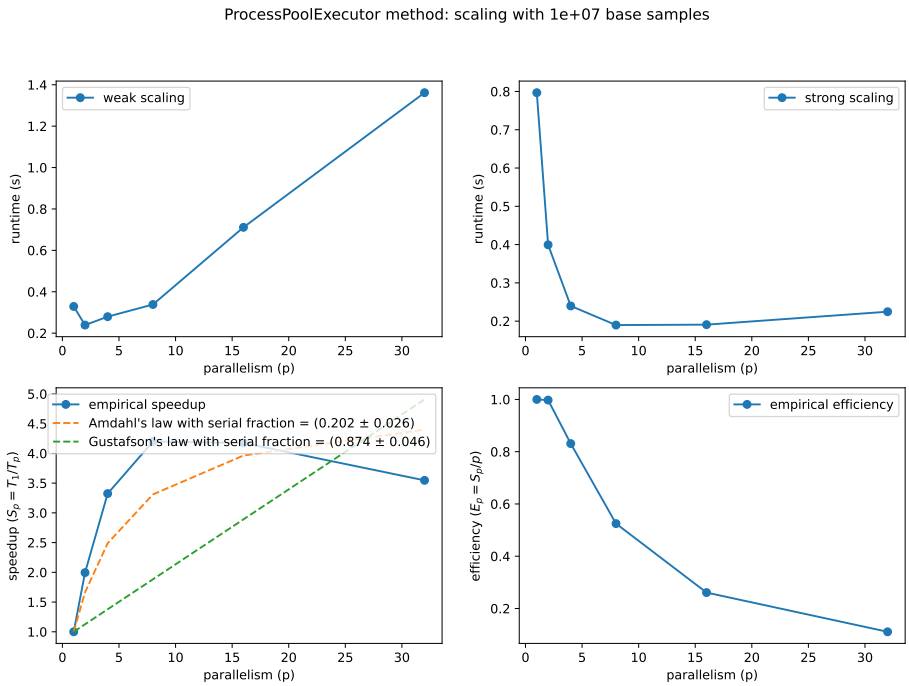


Figure 15: Scaling analysis for the ProcessPoolExecutor method. Speedup and efficiency correspond to strong scaling.

Concurrency	Speedup
1.0	1.0
2.0	1.9949645608322686
4.0	3.324090117303874
8.0	4.198623652886714
16.0	4.17520521248769
32.0	3.5465245557590177

Figure 16: Table of strong scaling data for the ProcessPoolExecutor method.

```python
async def add_chunk(seed, n, counts, u, x, cdf, bins=100, xmin=-10, xmax=10, n_subchunks=10):
    """
    Generate n samples in subchunks and add to global counts.
    """
    np.random.default_rng(seed)
    # Split n samples among sub-chunks
    sub_chunks = (n // n_subchunks) * np.ones(n_subchunks, dtype=int)
    sub_chunks[:n % n_subchunks] += 1 # Distribute remainder
    # Gather results from subchunks
    results = await asyncio.gather(*[
        async_lorentzian_histogram(chunk, bins, xmin, xmax)
        for chunk in sub_chunks
    ])
    # Unpack results
    for r in results:
        counts += r[0]
        u.extend(r[1])
        x.extend(r[2])
        cdf.extend(r[3])

async def get_counts(seeds, n, n_tasks=4, bins=100, xmin=-10, xmax=10, n_subchunks=10):
    """
    Async function to run the Lorentzian sampling in parallel using asyncio.
    """
    # Split n samples among tasks
    chunks = (n // n_tasks) * np.ones(n_tasks, dtype=int)
    chunks[:n % n_tasks] += 1 # Distribute remainder
    counts = np.zeros(bins) # Global counts
    u = [] # Global uniform samples
    x = [] # Global transformed samples
    cdf = [] # Global CDF
    tasks = [
        asyncio.create_task(add_chunk(seeds[i], chunks[i], counts, u, x, cdf, bins, xmin, xmax, n_subchunks))
        for i in range(n_tasks)
    ]
    await asyncio.gather(*tasks) # Wait for all tasks to finish
    return counts, u, x, cdf

def run_async(seeds, n, n_tasks=4, bins=100, xmin=-10, xmax=10, n_subchunks=10):
    """
    Run the Lorentzian sampling in parallel using asyncio.
    """
    return asyncio.run(get_counts(seeds, n, n_tasks, bins, xmin, xmax, n_subchunks))
```

(a)

```python
if __name__ == "__main__":

    method = 'AsyncIO'
    scaling = sys.argv[1] # 'weak' or 'strong'
    n = int(1e7) # Default number of samples
    n_tasks = 4 # Default number of tasks
    bins = 100 # Default number of bins
    xmin = -10 # Default xmin
    xmax = 10 # Default xmax
    n_subchunks = 10 # Default number of sub-chunks

    # Parse command line arguments, if provided
    if len(sys.argv) < 2:
        print("Usage: python async_lorentz.py <weak|strong> <n> <n_tasks> <bins> <xmin> <xmax> <n_subchunks>")
        print("Error: Scaling not specified.")
        sys.exit(1)
    if len(sys.argv) >= 3:
        n = int(float(sys.argv[2])) # Overwrite default n
    if len(sys.argv) >= 4:
        n_tasks = int(float(sys.argv[3])) # Overwrite default n_tasks
    if len(sys.argv) >= 5:
        bins = int(float(sys.argv[4])) # Overwrite default bins
    if len(sys.argv) >= 6:
        xmin = int(float(sys.argv[5])) # Overwrite default xmin
    if len(sys.argv) >= 7:
        xmax = int(float(sys.argv[6])) # Overwrite default xmax
    if len((sys.argv)) == 8:
        n_subchunks = int(float(sys.argv[7])) # Overwrite default n_subchunks
    if len(sys.argv) >= 9:
        print("Usage: python async_lorentz.py <weak|strong> <n> <n_tasks> <bins> <xmin> <xmax> <n_subchunks>")
        print("Additional arguments will be ignored.")

    seed_seq = np.random.SeedSequence(int(time()))
    seeds = seed_seq.spawn(n_tasks)

    start_time = perf_counter()
    counts, u, x, cdf = run_async(seeds, n, n_tasks, bins, xmin, xmax, n_subchunks)
    end_time = perf_counter()
    print(f'Method: {method}')
    print(f'Runtime: {end_time-start_time:ds}')
    with open(f"results/{scaling}_scaling_{method}.txt", "a", encoding="utf-8") as f:
        f.write(f"{n_tasks}\t{end_time-start_time}\n")
    if n_tasks == 1 and scaling == 'strong':
        plot_samples(method, counts, u, x, cdf, n, bins, xmin, xmax)
```

(b)

Figure 17: a: Modified AsyncIO algorithm. b: Main function.

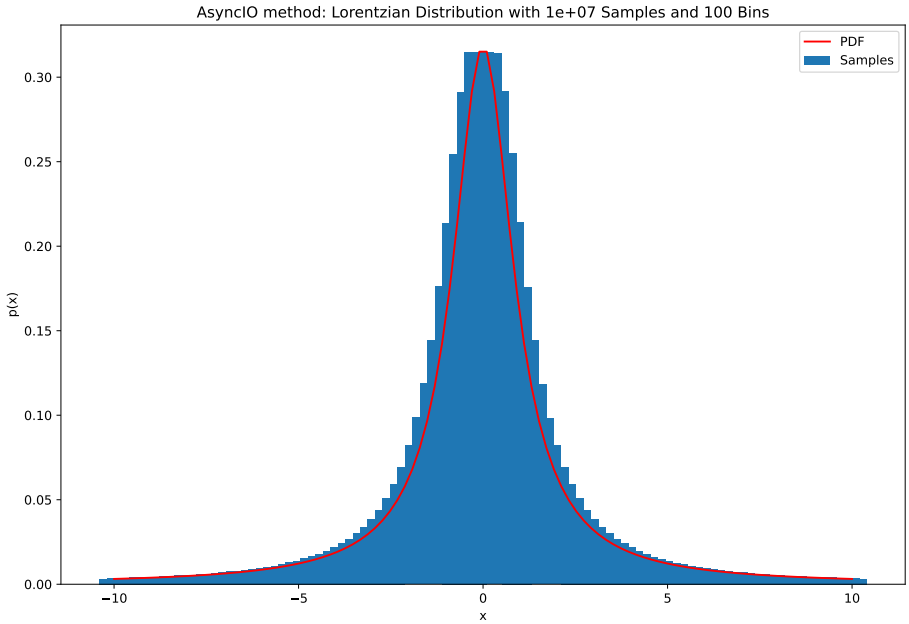


Figure 18: Sampling results from the AsyncIO method.

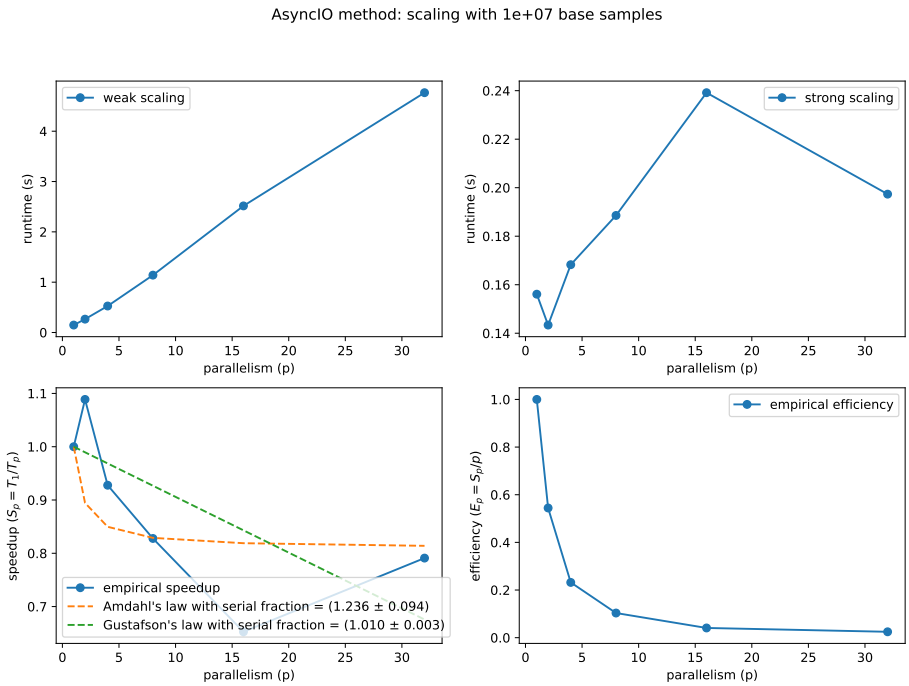


Figure 19: Scaling analysis for the AsyncIO method. Speedup and efficiency correspond to strong scaling.

Concurrency	Speedup
1.0	1.0
2.0	1.088765258993931
4.0	0.9277049503150006
8.0	0.827824478812257
16.0	0.6526540496353007
32.0	0.7909736691411123

Figure 20: Table of strong scaling data for the AsyncIO method.

```python
@delayed
def Delayed_lorentzian_histogram(seed, n, bins=100, xmin=-10, xmax=10):
    """
    Delayed function for lorentzian_histogram.
    """
    np.random.default_rng(seed)
    return lorentzian_histogram(n, bins, xmin, xmax)

def run_dask(seeds, n, n_tasks=4, bins=100, xmin=-10, xmax=10):
    """
    Run the Lorentzian sampling in parallel using Dask.
    """
    dask.config.set(scheduler='threads') # Override the default single-threaded scheduler
    # Split n samples among tasks
    chunks = [n // n_tasks] * np.ones(n_tasks, dtype=int)
    chunks[:n % n_tasks] += 1 # Distribute remainder
    tasks = [delayed_lorentzian_histogram(seeds[i], chunks[i], bins, xmin, xmax) for i in range(n_tasks)]
    results = dask.compute(*tasks) # Compute all tasks
    # Unpack results
    counts = np.zeros(bins)
    u = []
    x = []
    cdf = []
    for r in results:
        counts += r[0]
        u.extend(r[1])
        x.extend(r[2])
        cdf.extend(r[3])
    return counts, u, x, cdf
```

(a)

```python
if __name__ == "__main__":

    method = 'Dask'
    scaling = sys.argv[1] # 'weak' or 'strong'
    n = int(1e7) # Default number of samples
    n_tasks = 4 # Default number of tasks
    bins = 100 # Default number of bins
    xmin = -10 # Default xmin
    xmax = 10 # Default xmax

    # Parse command line arguments, if provided
    if len(sys.argv) < 2:
        print("Usage: python dask_lorentz.py <weak|strong> <n> <n_tasks> <bins> <xmin> <xmax> <n_subchunks>")
        print("Error: Scaling not specified.")
        sys.exit(1)
    if len(sys.argv) >= 3:
        n = int(float(sys.argv[2])) # Overwrite default n
    if len(sys.argv) >= 4:
        n_tasks = int(float(sys.argv[3])) # Overwrite default n_tasks
    if len(sys.argv) >= 5:
        bins = int(float(sys.argv[4])) # Overwrite default bins
    if len(sys.argv) >= 6:
        xmin = int(float(sys.argv[5])) # Overwrite default xmin
    if len(sys.argv) >= 7:
        xmax = int(float(sys.argv[6])) # Overwrite default xmax
    elif len(sys.argv) >= 8:
        print("Usage: python dask_lorentz.py <weak|strong> <n> <n_tasks> <bins> <xmin> <xmax>")
        print("Additional arguments will be ignored.")

    seed_seq = np.random.SeedSequence(int(time()))
    seeds = seed_seq.spawn(n_tasks)

    start_time = perf_counter()
    counts = run_dask(seeds, n, n_tasks, bins, xmin, xmax)
    end_time = perf_counter()
    print(f"Method: {method}")
    print(f"Runtime: {end_time-start_time:.6}s")
    with open(f"results/{scaling}_scaling_{method}.txt", "a", encoding="utf-8") as f:
        f.write(f"{n_tasks}\t{end_time-start_time}\n")
    if n_tasks == 1:
        plot_samples(method, counts, n, bins, xmin, xmax)
```

(b)

Figure 21: a: Modified Dask algorithm. b: Main function.

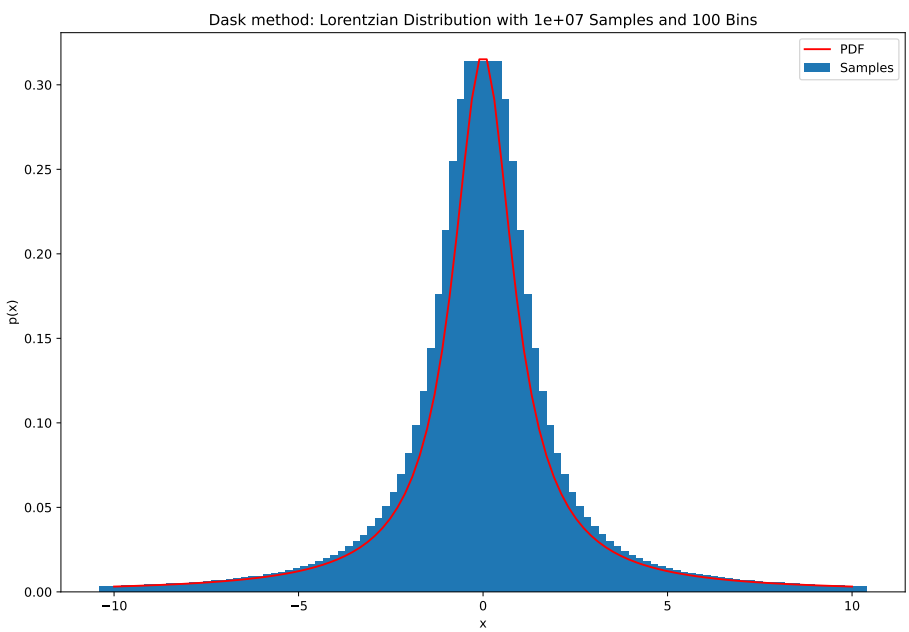


Figure 22: Sampling results from the Dask method.

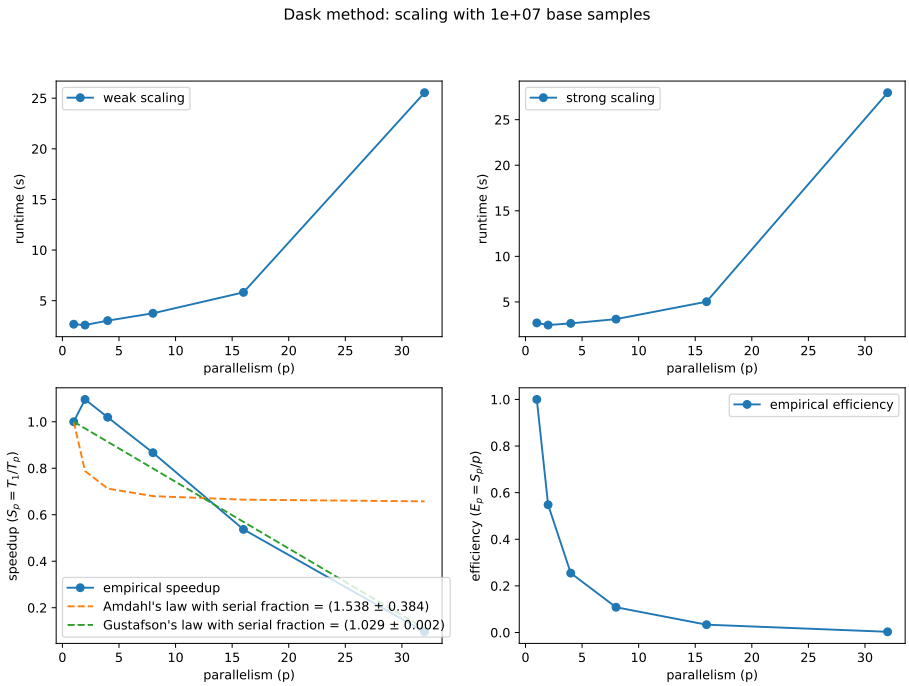


Figure 23: Scaling analysis for the Dask method. Speedup and efficiency correspond to strong scaling.

Concurrency	Speedup
1.0	1.0
2.0	1.096118040742851
4.0	1.0193597510583388
8.0	0.8668451534783116
16.0	0.5369267030908494
32.0	0.09645157766876233

Figure 24: Table of strong scaling data for the Dask method.

```python
# numba_lorentz.py
from numba import cuda
from numba.cuda.random import create_xoroshiro128p_states, xoroshiro128p_uniform_float32
import sys
import numpy as np
from time import time, perf_counter
from base_lorentz import plot_samples

@cuda.jit
def lorentzian_histogram_numba(rng_states, n, bins, xmin, xmax, counts, u, x, cdf):
    """
    Sample n random points from the Lorentzian distribution
    using inverse transform sampling with CUDA's random number generator.
    Make a histogram with the specified bin count and range. Modifies counts in-place.
    """
    thread_id = cuda.grid(1)
    stride = cuda.gridsize(1)  # Total number of threads
    idx = thread_id

    # Each thread processes multiple items with stride
    while idx < n:
        xfac = bins / (xmax - xmin)
        # Generate random number using this thread's RNG state
        local_u = xoroshiro128p_uniform_float32(rng_states, thread_id)
        u[idx] = local_u
        local_x = 1. / np.tan(np.pi * local_u)
        x[idx] = local_x
        local_cdf = np.arctan(local_x) / np.pi + 0.5
        cdf[idx] = local_cdf
        ix = int((local_x - xmin) * xfac)
        if 0 <= ix < bins:
            cuda.atomic.add(counts, ix, 1)

        idx += stride  # Move to next item assignad to this thread
```

(a)

```python
if __name__ == "__main__":

    method = 'numba'
    scaling = sys.argv[1] # 'weak' or 'strong'
    n = int(1e7) # Default number of samples
    blocks_per_grid = 1 # Default number of blocks
    threads_per_block = 4 # Default number of threads
    bins = 100 # Default number of bins
    xmin = -10 # Default xmin
    xmax = 10 # Default xmax

    # Parse command line arguments, if provided
    if len(sys.argv) < 2:
        print("Usage: python numba_lorentz.py <weak|strong> <n> <n_tasks> <bins> <xmin> <xmax> <n_subchunks>")
        print("Error: Scaling not specified.")
        sys.exit(1)
    if len(sys.argv) >= 3:
        n = int(float(sys.argv[2])) # Overwrite default n
    if len(sys.argv) >= 4:
        blocks_per_grid = int(float(sys.argv[3])) # Overwrite default blocks_per_grid
    if len(sys.argv) >= 5:
        threads_per_block = int(float(sys.argv[4])) # Overwrite default threads_per_block
    if len(sys.argv) >= 6:
        bins = int(float(sys.argv[5])) # Overwrite default bins
    if len(sys.argv) >= 7:
        xmin = int(float(sys.argv[6])) # Overwrite default xmin
    if len(sys.argv) == 8:
        xmax = int(float(sys.argv[7])) # Overwrite default xmax
    if len(sys.argv) >= 9:
        print("Usage: python numba_lorentz.py <weak|strong> <n> <bins> <xmin> <xmax>")
        print("Additional arguments will be ignored.")

    start_time = perf_counter()

    # Create per-thread random number states for the GPU
    num_threads = blocks_per_grid * threads_per_block
    rng_states = create_xoroshiro128p_states(num_threads, seed=int(time()))

    # Allocate output arrays
    counts = np.zeros(bins, dtype=np.int32)
    u = np.zeros(n, dtype=np.float32)
    x = np.zeros(n, dtype=np.float32)
    cdf = np.zeros(n, dtype=np.float32)

    # Launch kernel with RNG states as first argument
    lorentzian_histogram_numba[blocks_per_grid, threads_per_block](rng_states, n, bins, xmin, xmax, counts, u, x, cdf)
```

(b)

Figure 25: a: Modified Numba algorithm. b: Main function.

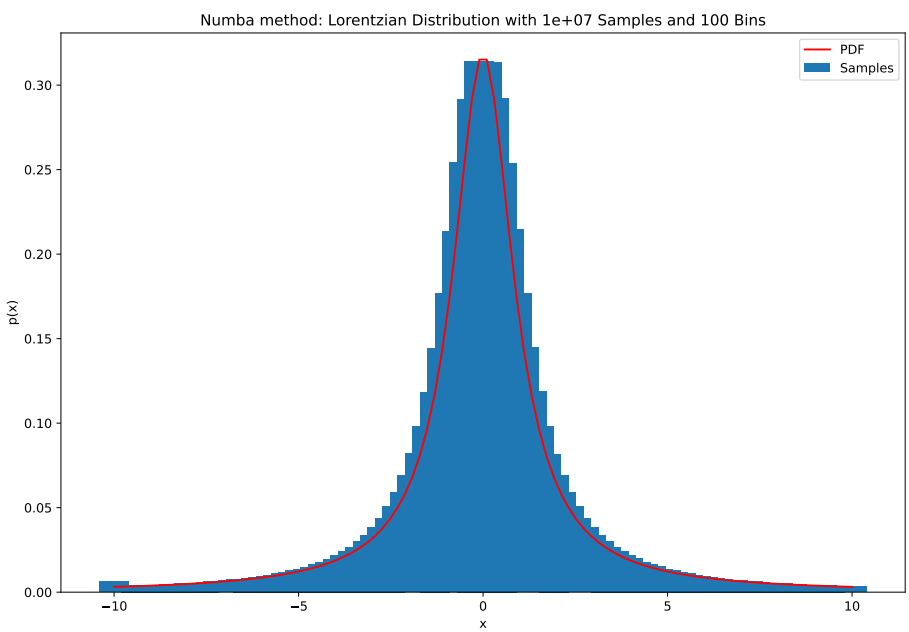


Figure 26: Sampling results from the Numba method.

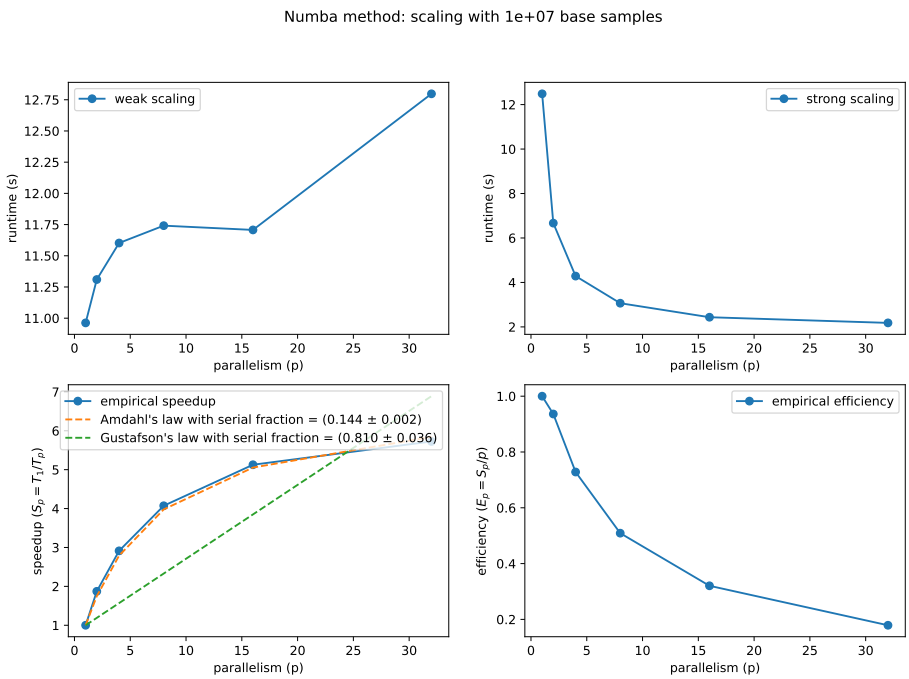


Figure 27: Scaling analysis for the Numba method. Speedup and efficiency correspond to strong scaling.

Concurrency	Speedup
1.0	1.0
2.0	1.8723402048976177
4.0	2.9130402522094307
8.0	4.072148292960055
16.0	5.126515970892372
32.0	5.725416587685006

Figure 28: Table of strong scaling data for the Numba method.

```python
def local_job(seed, n, bins=100, xmin=-10, xmax=10):
    np.random.default_rng(seed)
    return lorentzian_histogram(n, bins, xmin, xmax)

def run_joblib(seeds, n, n_jobs=4, bins=100, xmin=-10, xmax=10):
    """
    Run the Lorentzian sampling in parallel using joblib.
    """
    # Split n samples among jobs
    chunks = (n // n_jobs) * np.ones(n_jobs, dtype=int)
    chunks[:n % n_jobs] += 1 # Distribute remainder
    results = Parallel(n_jobs=n_jobs)(
        delayed(local_job)(seeds[i], chunks[i], bins, xmin, xmax)
        for i in range(n_jobs)
    )
    # Unpack results
    counts = np.zeros(bins)
    u = []
    x = []
    cdf = []
    for r in results:
        counts += r[0]
        u.extend(r[1])
        x.extend(r[2])
        cdf.extend(r[3])
    return counts, u, x, cdf
```

(a)

```python
if __name__ == "__main__":

    method = 'Joblib'
    scaling = sys.argv[1] # 'weak' or 'strong'
    n = int(1e7) # Default number of samples
    n_jobs = 4 # Default number of jobs
    bins = 100 # Default number of bins
    xmin = -10 # Default xmin
    xmax = 10 # Default xmax

    # Parse command line arguments, if provided
    if len(sys.argv) < 2:
        print("Usage: python joblib_lorentz.py <weak|strong> <n> <n_tasks> <bins> <xmin> <xmax> <n_subchunks>")
        print("Error: Scaling not specified.")
        sys.exit(1)
    if len(sys.argv) >= 3:
        n = int(float(sys.argv[2])) # Overwrite default n
    if len(sys.argv) >= 4:
        n_jobs = int(float(sys.argv[3])) # Overwrite default n_jobs
    if len(sys.argv) >= 5:
        bins = int(float(sys.argv[4])) # Overwrite default bins
    if len(sys.argv) >= 6:
        xmin = int(float(sys.argv[5])) # Overwrite default xmin
    if len(sys.argv) == 7:
        xmax = int(float(sys.argv[6])) # Overwrite default xmax
    elif len(sys.argv) >= 8:
        print("Usage: python joblib_lorentz.py <weak|strong> <n> <n_jobs> <bins> <xmin> <xmax>")
        print("Additional arguments will be ignored.")


    seed_seq = np.random.SeedSequence(int(time()))
    seeds = seed_seq.spawn(n_jobs)


    start_time = perf_counter()
    counts = run_joblib(seeds, n, n_jobs, bins, xmin, xmax)
    end_time = perf_counter()
    print(f'Method: {method}')
    print(f'Runtime: {end_time-start_time:.6}s')
    with open(f"results/{scaling}_scaling_{method}.txt", "a", encoding="utf-8") as f:
        f.write(f"{n_jobs}\t{end_time-start_time}\n")
    if n_jobs == 1:
        plot_samples(method, counts, n, bins, xmin, xmax)
```

(b)

Figure 29: a: Modified Joblib algorithm. b: Main function.

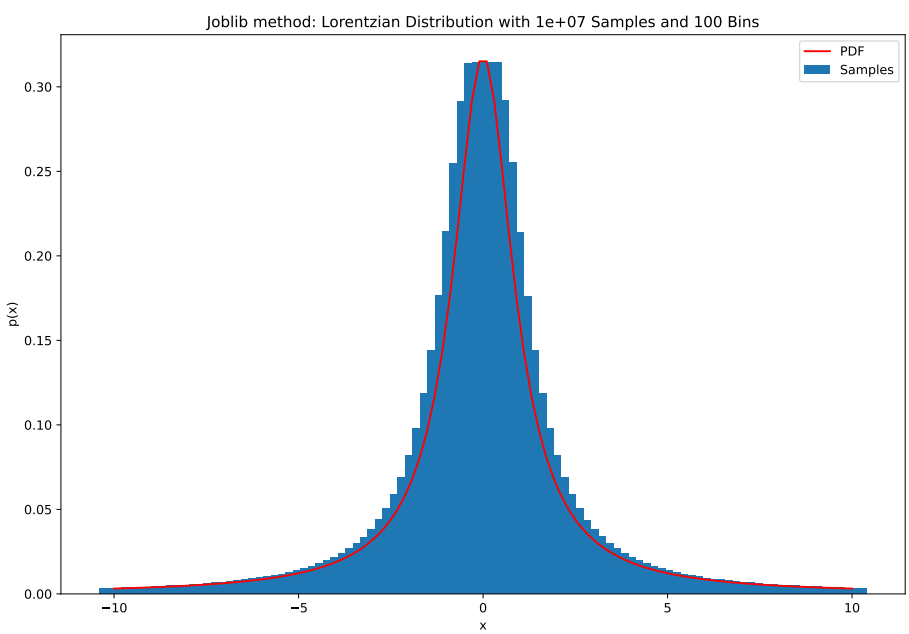


Figure 30: Sampling results from the Joblib method.

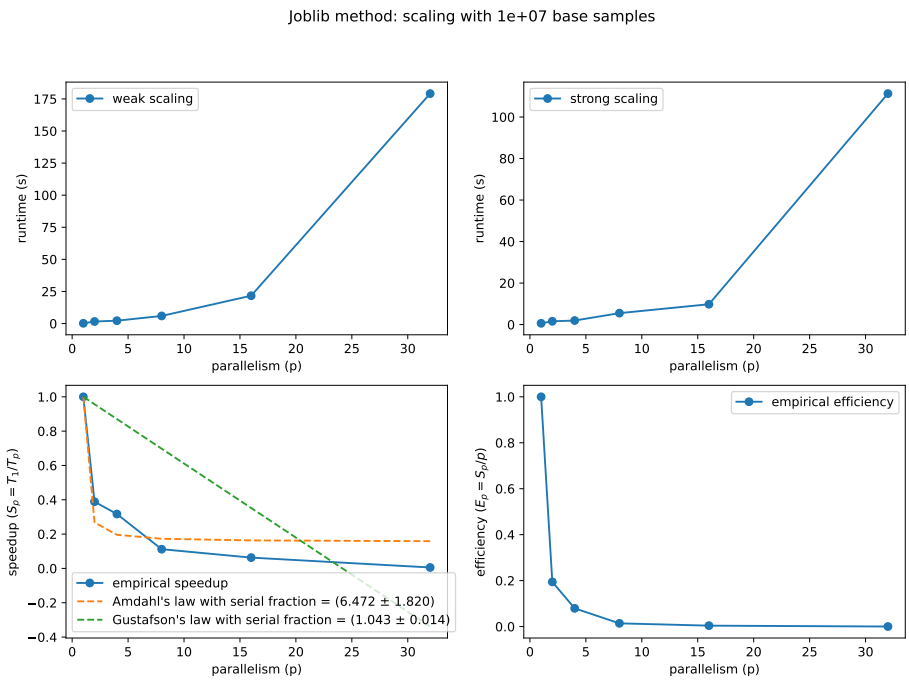


Figure 31: Scaling analysis for the Joblib method. Speedup and efficiency correspond to strong scaling.

Concurrency	Speedup
1.0	1.0
2.0	0.3888106132089739
4.0	0.31695970274599544
8.0	0.1120026913977484
16.0	0.06299503392627631
32.0	0.005547041356249362

Figure 32: Table of strong scaling data for the Joblib method.

```python
# mpire_lorentz.py
import sys
from mpire import WorkerPool
import numpy as np
from time import time, perf_counter
from base_lorentz import lorentzian_histogram, plot_samples

def init_func(worker_state):
    # Initialize rng and counts for each worker
    worker_state['seed'] = int(time())

def mpire_lorentzian_histogram(worker_state, n, bins=100, xmin=-10, xmax=10):
    np.random.default_rng(worker_state['seed'])
    worker_state['counts'] = lorentzian_histogram(n, bins, xmin, xmax)

def exit_func(worker_state):
    # Return counts
    return worker_state['counts']

def run_mpire(n, n_jobs=4, bins=100, xmin=-10, xmax=10):
    """
    Run the Lorentzian sampling in parallel using mpire.
    """
    # Split n samples among jobs
    chunks = (n // n_jobs) * np.ones(n_jobs, dtype=int)
    chunks[:n % n_jobs] += 1 # Distribute remainder
    with WorkerPool(n_jobs=n_jobs, use_worker_state=True) as pool:
        # See mpire docs for argument passing; alternatively use starmap
        # pool.set_initializer(initializer)
        pool.map(mpire_lorentzian_histogram, chunks, worker_init=init_func, worker_exit=exit_func)
        counts = pool.get_exit_results()
    if n_jobs > 1:
        counts = np.sum(counts, axis=0)
    return counts
```

(a)

```python
if __name__ == "__main__":

    method = 'mpire'
    scaling = sys.argv[1] # 'weak' or 'strong'
    n = int(1e7) # Default number of samples
    n_jobs = 4 # Default number of jobs
    bins = 100 # Default number of bins
    xmin = -10 # Default xmin
    xmax = 10 # Default xmax

    # Parse command line arguments, if provided
    if len(sys.argv) < 2:
        print("Usage: python mpire_lorentz.py <weak|strong> <n> <n_tasks> <bins> <xmin> <xmax> <n_subchunks>")
        print("Error: Scaling not specified.")
        sys.exit(1)
    if len(sys.argv) >= 3:
        n = int(float(sys.argv[2])) # Overwrite default n
    if len(sys.argv) >= 4:
        n_jobs = int(float(sys.argv[3])) # Overwrite default n_jobs
    if len(sys.argv) >= 5:
        bins = int(float(sys.argv[4])) # Overwrite default bins
    if len(sys.argv) >= 6:
        xmin = int(float(sys.argv[5])) # Overwrite default xmin
    if len(sys.argv) == 7:
        xmax = int(float(sys.argv[6])) # Overwrite default xmax
    elif len(sys.argv) >= 8:
        print("Usage: python mpire_lorentz.py <weak|strong> <n> <n_jobs> <bins> <xmin> <xmax>")
        print("Additional arguments will be ignored.")

    start_time = perf_counter()
    counts = run_mpire(n, n_jobs, bins, xmin, xmax)
    end_time = perf_counter()
    print(f'Method: {method}')
    print(f'Runtime: {end_time-start_time:.6}')
    with open(f"results/{scaling}_scaling_{method}.txt", "a", encoding="utf-8") as f:
        f.write(f"{n_jobs}\t{end_time-start_time}\n")
    if n_jobs == 2:
        plot_samples(method, counts, n, bins, xmin, xmax)
```

(b)

Figure 33: a: Modified mpire algorithm. b: Main function.

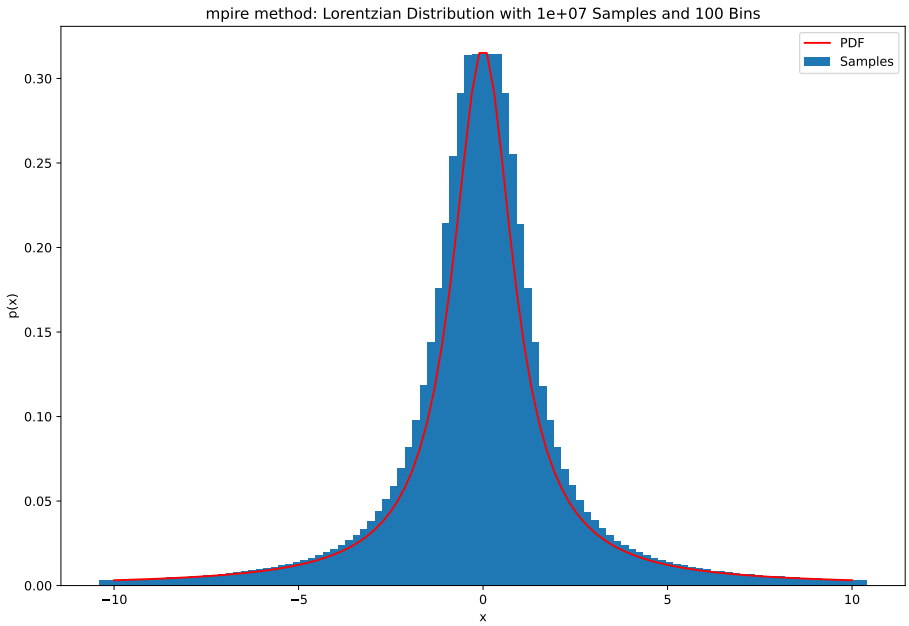


Figure 34: Sampling results from the mpire method.

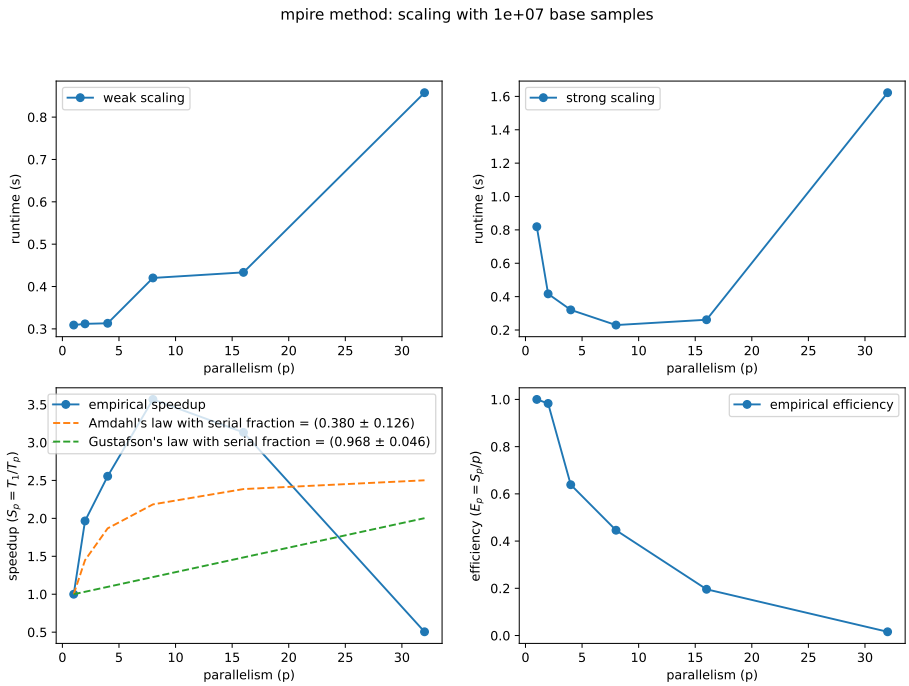


Figure 35: Scaling analysis for the mpire method. Speedup and efficiency correspond to strong scaling.

Concurrency	Speedup
1.0	1.0
2.0	1.965972759284937
4.0	2.554846404059422
8.0	3.5674991653311254
16.0	3.133149827176136
32.0	0.5049026195760443

Figure 36: Table of strong scaling data for the mpire method.

```python
# mpi_lorentz.py
import sys
import numpy as np
from mpi4py import MPI
from time import time, perf_counter
from base_lorentz import plot_samples

def lorentzian_histogram(n, bins=100, xmin=-10, xmax=10, rng=None):
    if rng is None:
        rng = np.random.default_rng()
    u = rng.random(n)
    x = 1. / np.tan(np.pi * u)
    counts, _ = np.histogram(x, bins=bins, range=(xmin, xmax))
    return counts.astype(np.int64)

scaling = sys.argv[1] # 'weak' or 'strong'
n_total = 10_000_000
bins = 100
xmin = -10
xmax = 10

# Parse command line arguments, if provided
if len(sys.argv) < 2:
    print("Usage: python mpi_lorentz.py <weak|strong> <n> <n_tasks> <bins> <xmin> <xmax> <n_subchunks>")
    print("Error: Scaling not specified.")
    sys.exit(1)
if len(sys.argv) >= 3:
    n = int(float(sys.argv[2])) # Overwrite default n
if len(sys.argv) >= 4:
    bins = int(float(sys.argv[3])) # Overwrite default bins
if len(sys.argv) >= 5:
    xmin = int(float(sys.argv[4])) # Overwrite default xmin
if len(sys.argv) == 6:
    xmax = int(float(sys.argv[5])) # Overwrite default xmax
if len(sys.argv) >= 7:
    print("Usage: python mpi_lorentz.py <weak|strong> <n> <n_cores> <bins> <xmin> <xmax>")
    print("Additional arguments will be ignored.")
```

(a)

```python
comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

# Start timing
if rank == 0:
    start_time = perf_counter()

# Independent RNG stream per rank
seed = int(time())
ss = np.random.SeedSequence(seed)
child = ss.spawn(size)[rank]
rng = np.random.default_rng(child)

chunks = np.full(size, n_total // size, dtype=int)
chunks[: n_total % size] += 1
local = lorentzian_histogram(int(chunks[rank]), bins=bins, xmin=xmin, xmax=xmax, rng=rng)
global_counts = np.empty_like(local)
comm.Allreduce(local, global_counts, op=MPI.SUM)

if rank == 0:
    end_time = perf_counter()
    print(f"Total samples: {n_total}")
    print(f"Runtime: {end_time - start_time:.3f} seconds")
    print(f"Samples per second: {n_total / (end_time - start_time):.0f}")

    # Save results
    bin_edges = np.linspace(-10, 10, 101)
    bin_centers = (bin_edges[:-1] + bin_edges[1:]) / 2
    np.savetxt("lorentzian_histogram.txt",
    np.column_stack([bin_centers, global_counts]),
    fmt="%.6f %d")
    print("Results saved to lorentzian_histogram.txt")
    with open(f"results/{scaling}_scaling_MPI.txt", "a", encoding="utf-8") as f:
        f.write(f"{size}\t{end_time-start_time}\n")
    if size == 1:
        plot_samples('MPI', global_counts, n_total, bins, xmin, xmax)
```

(b)

Figure 37: a: Modified MPI algorithm. b: Main function.

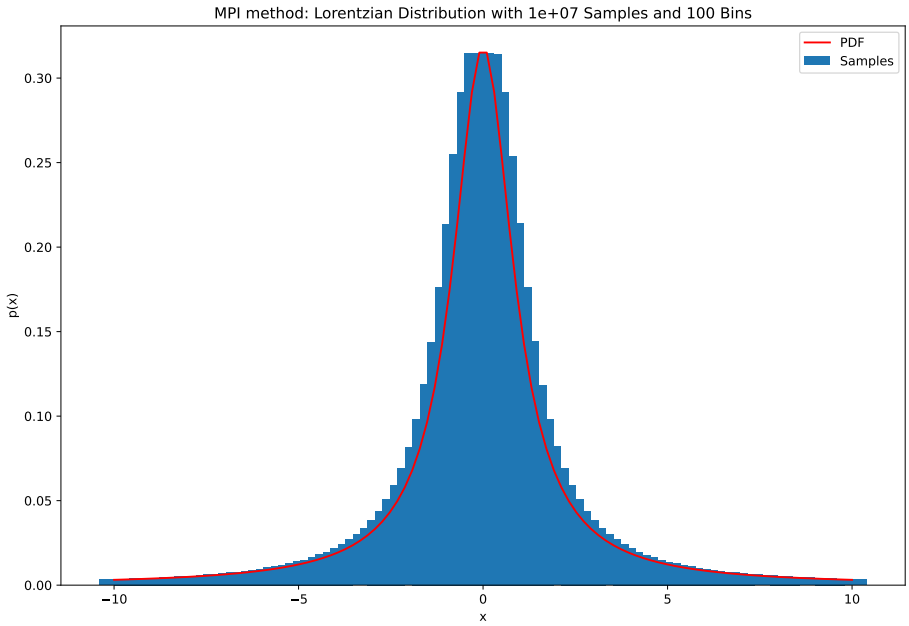


Figure 38: Sampling results from the MPI method.

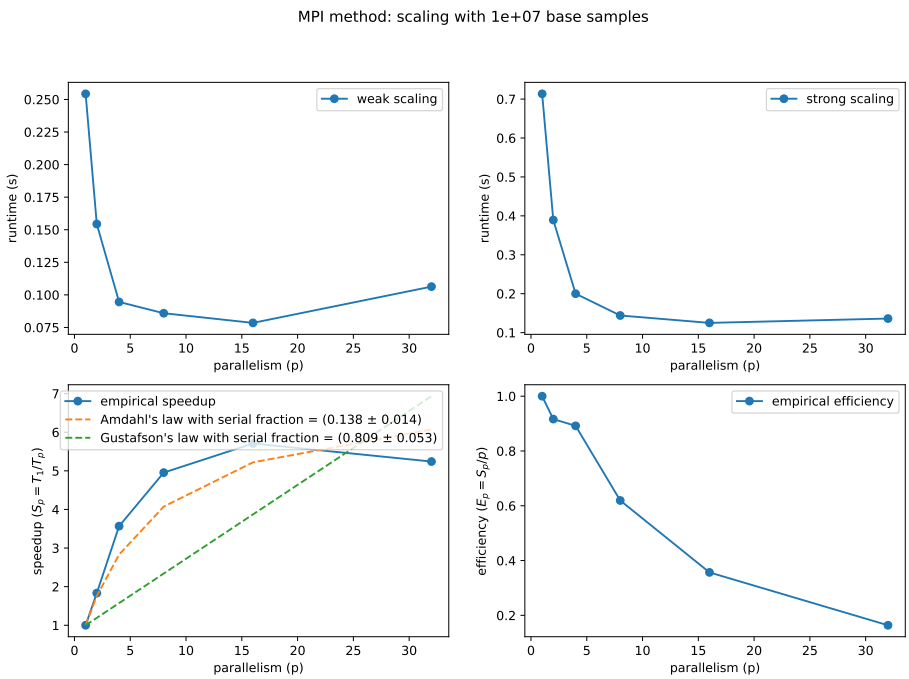


Figure 39: Scaling analysis for the MPI method. Speedup and efficiency correspond to strong scaling.

Concurrency	Speedup
1.0	1.0
2.0	1.8328799265896298
4.0	3.566984441242601
8.0	4.954097564340143
16.0	5.707651660384745
32.0	5.240780586458347

Figure 40: Table of strong scaling data for the MPI method.

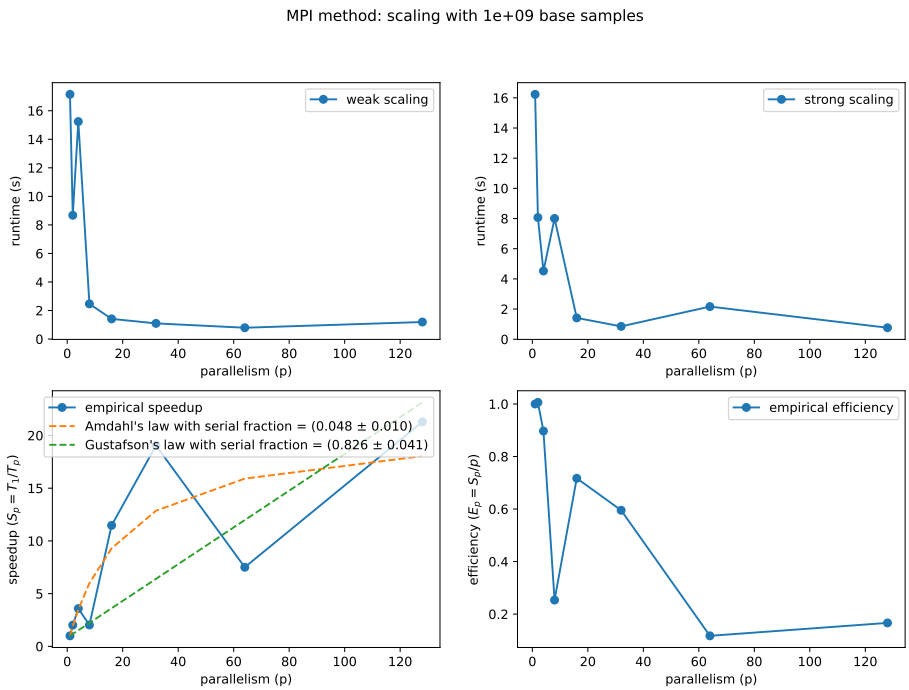


Figure 41: Scaling analysis for the MPI method. Speedup and efficiency correspond to strong scaling.

Concurrency	Speedup
1.0	1.0
2.0	2.0120059056796777
4.0	3.5882958053665694
8.0	2.0272409056948772
16.0	11.474898882570077
32.0	19.049470205000826
64.0	7.502070106307589
128.0	21.29420918950063

Figure 42: Table of strong scaling data for the MPI method.